Khedmni Application Feature Roadmap (Full Epic Breakdown)

Project Planning & Organization

December 7, 2025

Introduction

This document refines the development roadmap for the Khedmni application, organizing all features into four key Epics. Each Epic includes a Definition of Done (DoD), proposed free technology stack, and detailed subtasks for implementation, providing a complete project management view.

1. Epic 1: Core Authentication & Security

Focus: User lifecycle, security foundation, and application theming.

1.1. Definition of Done (DoD)

- New user sign-up flow correctly enforces email uniqueness and initiates a verification email.
- User cannot create a profile until the email is successfully verified.
- Company profile model includes fields for verification documents and the admin can toggle the ”Verified” status.
- A global Settings page exists, allowing the user to toggle between Light and Dark mode, which applies consistently across the app.

1.2. Free Technology Recommendation

- **Backend/Auth: Firebase Authentication** (handles email uniqueness, verification email service, and user state for free tier) or **Supabase** (PostgreSQL + Auth).
- **Database: Firestore** or **Supabase (PostgreSQL)**.
- **Frontend:** Standard web framework (React/Angular) or **Flutter** for mobile, styled with responsive CSS/Tailwind classes.

1.3. Subtasks

1.3.1. Security & Auth (Tasks 1, 2, 3)

1. Implement a unique index/constraint on the email field in the user database collection.
2. Create the ”Email Verification Sent” interstitial screen on the frontend.
3. Configure Firebase Auth/Backend to send verification emails upon sign-up.
4. Implement a logic gate that prevents navigation from the verification screen to the ”Create Profile” screen until the email verification status is confirmed.
5. Add document upload fields to the Company profile form.
6. Create a simple, secure admin endpoint to receive company verification documents (license, etc.).

1.3.2. UI/UX & Theming (Tasks 8, 9)

1. Create the basic structure and navigation for the **Settings Page**.
2. Define two distinct theme objects (Light and Dark) using colors from `colors.dart`.
3. Implement a global theme provider/context to manage the current theme state.

4. Add a toggle switch in the Settings page to persist and apply the selected theme preference.

2. Epic 2: Job & Internship Core Management

Focus: The ability to search, post, manage, and apply to job and internship opportunities.

2.1. Definition of Done (DoD)

- Users can successfully upload a CV/Resume and submit it with an application.
- All job and internship posts are clearly differentiated via tags/filters.
- The search bar returns highly relevant results based on title, description, and tags.
- Companies can save and retrieve job posts as drafts.
- Users can save jobs to their personal profile section.

2.2. Free Technology Recommendation

- **Storage: Firebase Storage** or **AWS S3** (free tier for file uploads like CVs).
- **Search: PostgreSQL** with built-in full-text search capabilities, or utilize Firebase's indexed queries.

2.3. Subtasks

2.3.1. Application & Posting Management (Tasks 11, 13)

1. Implement the secure file upload service for PDF/DOCX files (CVs).
2. Update the `Application` schema to include the CV file URL.
3. Add a `status` field (`DRAFT`, `ACTIVE`, `CLOSED`) to the `JobPost` model.
4. Modify the Company Post form to save posts with the `DRAFT` status without requiring all mandatory fields.

2.3.2. Internship & Search (Tasks 7, 4, 17)

1. Update the `JobPost` schema with a mandatory `isInternship` boolean and an optional `duration` field.
2. Implement a dedicated "Internships" view or filter component on the main job feed.
3. Configure the backend search query to perform efficient full-text matching across `title`, `description`, and `tags`.
4. Create a `SavedJobs` collection/table, mapping `userId` to `jobPostId`.
5. Implement the "Save" button UI and persistence logic on the job detail screen.

3. Epic 3: Engagement, Notifications & UX

Focus: Social features, real-time feedback, and polishing the user-facing job feed.

3.1. Definition of Done (DoD)

- The main Job Feed UI is aesthetically modern, responsive, and easy to navigate.
- The Notification system is fully functional, with all defined triggers sending notifications to the correct user/company profiles.
- Users can follow any profile, and a dedicated feed section aggregates updates from followed entities.

3.2. Free Technology Recommendation

- **UI/UX: Tailwind CSS** utility-first framework for rapid, responsive design.
- **Real-Time/Notifications: Firebase Cloud Messaging (FCM)** or **WebSockets** (if using a custom backend) for delivering real-time alerts.
- **Database: Firestore** for real-time 'onSnapshot' listeners on the notifications collection.

3.3. Subtasks

3.3.1. UI/UX Redesign (Task 5)

1. Develop a new, modern Card Component for job listings, emphasizing clear hierarchy and using whitespace effectively.
2. Apply responsive grid/flex layouts to the homepage to ensure appeal on all screen sizes.

3.3.2. Notification System (Task 6)

1. Create the `Notifications` database model (fields: `userId`, `type`, `relatedId`, `isRead`, `timestamp`).
2. Implement backend functions to trigger notifications on: Application Submitted (to Company) and Application Status Change (to User).
3. Implement the frontend Notification Bell UI, displaying the unread count.

3.3.3. Follower System (Tasks 15, 16)

1. Create the `Followers` database model (mapping `followerId` to `followedId`).
2. Implement the "Follow" button UI component on User and Company profiles, with instant count updates.
3. Develop the backend query logic for the "Follower Updates Feed," aggregating recent posts from all followed profiles.

4. Epic 4: Company Portal & Monetization

Focus: Enhancing tools for companies, providing analytics, and implementing premium features.

4.1. Definition of Done (DoD)

- Company profile is enhanced with rich content (gallery/videos) and displays the verification badge.
- Companies have a dashboard showing application statistics (1, 7, 30-day views).
- Companies can filter applicants by job tags/skills.

- The Premium AI ranking feature is implemented, secured behind a paywall, and provides a ranked list of top applicants.

4.2. Free Technology Recommendation

- **Analytics/Reporting:** Standard database aggregation queries (e.g., `GROUP BY` and `COUNT`) for reporting volumes.
- **AI/ML Ranking (Premium):** Integrate a general-purpose **Gemini API** endpoint (or similar external service) to score CV text against job description text.
- **Payments:** Integrate a service like **Stripe** or **PayPal** (though these will involve transaction fees, basic integration is free).

4.3. Subtasks

4.3.1. Applicant Management (Tasks 10, 18)

1. Implement a **basic applicant score** based on shared tags/skills between the applicant's profile and the job requirements.
2. Display the resulting top N applicants in the Company Portal job view.
3. Build the frontend filter components (checkboxes/dropdowns) in the applicant view.
4. Implement the backend query to filter applicants by selected tags/skills.

4.3.2. Analytics & Enhancement (Tasks 12, 14)

1. Implement backend functions to calculate and return application counts for the defined time periods (1, 7, 30 days, year).
2. Develop the Company Post Management dashboard UI to display the calculated statistics.
3. Update the Company profile model to include fields for a `galleryURLs` array and `teamMembers` list.
4. Implement the logic to display the "Verified" badge based on the status set in Epic 1.

4.3.3. Monetization & Integration (Tasks 19, 20)

1. Implement the payment gateway integration (Task 20) for handling subscriptions.
2. Research and select an NLP-based API for advanced CV matching (Task 19).
3. Create a secure backend endpoint to call the ranking API, passing the job description and applicant CV text.
4. Implement the **paywall logic** that checks a company's subscription status before displaying the AI-ranked list.